

SQL - Single Table Queries

[] = optional

SELECT [DISTINCT] <columns>
FROM <table>
[WHERE <predicate>]
[GROUP BY <columns>]
[HAVING <predicate>]
[ORDER BY <columns> [DESC/ASC]]
[LIMIT <amount>]

Logical Processing Order

1. FROM <table> - table we are drawing data from
2. WHERE <pred> - only keep rows where <pred> is satisfied
3. GROUP BY <columns> - group together rows by their values of <columns>
4. HAVING <pred> - only keep groups that satisfy <pred>
5. SELECT <columns> - select columns to keep
6. DISTINCT - keep only distinct rows
7. ORDER BY <columns> [DESC/ASC] - order output by value of <columns>, ASC by default
8. LIMIT <amount> - limit output to the first <amount> rows

Columns can only be selected if they are part of the GROUP BY clause or it's an aggregate (AVG, MIN, MAX, SUM, COUNT, etc.)

LIKE can be used for <pred>
_: Any single character
%: zero or more characters

LIKE 'exp', LIKE '%2%', etc.

INNER JOINS

- Return rows where the ON condition is true

LEFT AND RIGHT JOINS

Each row in the left table appears, with any matching rows in the right table

Vice Versa for Right Joins

View Serializability

View Serializability $\subseteq$ Conflict Serializability $\subseteq$ Serializability.

- Conflict Serializability but blind writes can be reordered.
- Blind writes: writes with no reads in between.

Checks: ① Same initial reads - same transaction reads the initial value of each resource.

② Same dependent reads - each read gets its value from the same transaction in both schedules. If T1 reads from T2's write in S1, it must do the same in S2.

③ Same winning writes - same transaction writes the final value of each resource.

Phantom Problem is when the same query returns different rows within the same transaction due to concurrent insert/delete.

FULL JOINS

- Returns all rows from both tables, matching rows are combined, non matching rows appear w/ nulls.

SQL Tips

- No aggregates in WHERE
- Use HAVING only with GROUP BY, use WHERE otherwise
- If using GROUP BY, can only SELECT aggregate columns or columns in the GROUP BY.
- DISTINCT removes all duplicate rows

Page/Record Formats

- Tables are stored as logical files consisting of pages, each of which contains a collection of records.

Pages and I/Os

- Pages are managed
 - in memory by the buffer manager: higher levels of database only operate in memory
 - in disk by the disk space manager: reads and writes pages to physical disk/files

- The unit of accesses to physical disk is the page.
- cannot fetch fractions of a page.

- I/O: unit of transferring a page of data between memory and disk (read or write 1 page = 1 I/O).

Data Types

Fixed or Var?	Data Type	Size
Fixed	Integer	4B
Fixed	float	4B
Fixed	boolean	1B
Fixed	char(20)	20B
Fixed	byte	1B
Variable	text	$\geq 0B$
Variable	varchar(20)	$\leq 20B$

1 bit for every field that can be null, round to bytes.

Ptr = 4 bytes

Fixed Length Records

- Records made up of multiple fields
 - fixed length records are when field lengths are consistent
- Ex: Students (sid integer PRIMARY KEY, enrolled boolean, grad-yr integer);

Record1 = 1234 | True | 2023

Storing FLRs

- packed: no gaps between records, records placed continuously one after the other
- unpacked: allows gaps between records, use a bitmap to keep track of where the gaps are.

Variable Length Records

- field lengths not consistent
- record lengths not fixed

Ex: Students (sid integer PRIMARY KEY, name text, grad-yr integer);

Record1 = 1234 | Ben | 2023

Storing VLRs

- Each record contains a record header
- Variable length fields are placed after fixed length fields
- Record header stores field offset indicating where each variable length field ends.

Pages

Page header - portion of each page reserved to keep track of the records in the page.

- # records in page (4B)
- pointer to segment of free space in the page (4B)
- bitmap indicating which parts of the page are in use

Accessing Disk

seek time = 2-3 m/s
rot delay = 0-4 m/s
transfer time = 0.25 m/s

SSDs have limited erasures for writes

Locking

S lock - lets a transaction read a resource. Many transactions can hold S locks on a resource at once.

X lock - lets a transaction modify a resource. No other transaction can have any type of lock while a transaction has an X lock.

If conflicting lock, pause and wait until the other transaction releases the conflicting lock.

Deadlock is when a bunch of transactions are waiting on each other in a cycle

Deadlock avoidance: T1 wants a lock but T2 holds a conflicting lock. wait-die: if T1 higher priority, T1 waits for T2 to release lock. if T1 lower priority, T1 aborts

wound-waits: if T1 higher priority, it causes T2 to abort. if T1 lower priority, it waits for T2 to finish

Priority by age: now - start time

Deadlock Detection: Build graph, node per transaction, if T1 holds a lock that conflicts with the lock that T2 wants, add edge from T2 to T1. Cycle indicates deadlock. Abort one to end deadlock.

VLRs: Slotted Pages

- move page header to be a footer to allow space for records to grow
- store length and pointer to start of each record in footer (4B each)
- Store #slots and ptr to free space

Deleting Records

- Unpacked Layout: nothing changes, but there is empty slot
 - can cause fragmentation
 - 2 free bytes in one place, 2 more else.Since slots are kept, cannot insert 4 byte record.
- Packed Layout: slot removed, record length, ptr pair removed, free space ptr updated, slot count decreases

File Organization

- Heap file is a file w/ no order enforced. Records are placed arbitrarily across pages
- Sorted file is sorted on a key (subset of the fields)

Implementing Heap Files

- Linked list: each page has 2 ptrs: free space and data. You have 2 linked lists of pages, both connected to a header page.
 - List of full pages
 - List of pages w/ free space.
- Page directory: linked list of header pages. Each entry of a header page contains a ptr to a data page, and amt of free space for the data page.

If implementation given, include I/O cost of reading/writing the file's header pages
2 I/Os per modified page

2 Phase Locking (2PL)

- From start to until a lock is released, the transaction only acquires locks. Then until the end, it only releases locks.

Cascading aborts can occur if a transaction reads an uncommitted values which are later aborted. All transactions that directly or indirectly read its data must also abort.

Strict 2PL avoids this by only allowing locks to be released at the end of the transaction, when it commits.

Indices

- DS that allows for fast lookup on a search key
- lookup can be equality, 1D, 2D, etc.
- search key is a subset of columns
- may store pointers to heap file (key, recordID) or entire records

B+ trees $O(h)$ to find key

- Leaf nodes contain data entries (key, recordID)
- Inner nodes are only for lookup, left is <, right is $\geq$
- d = order of tree
- Each node contains up to $2d$ values
- Height = # of pointers from root to a leaf node.
- Max fanout = $2d+1$
- Occupancy Invariant: every inner node (except root) must be at least half full
- may also add sibling ptrs between leaf nodes

Insertion

1. Find correct leaf L
2. Put data entry into L
3. If enough space, done!
Else, split into 2 nodes with d and $d+1$ entries.
4. Copy leftmost entry of right child to parent node and add references.
5. If inner node full, split in 2 and push middle val up

Max # data entries:

$$\underbrace{2d}_{\text{node cap}} \cdot \underbrace{(2d+1)^h}_{\text{max fanout}}$$

Bulkloading

When fill factor exceeded, add to parent node, and create new node and add there also. If inner node full (not fill factor), split and send up middle val. Adjust ptrs as needed.

Data Storage

Alt 1: store actual records

Alt 2: store <key, recordID>

Alt 3: store <key, list of matching recordIDs>

RecordID is page #, slot #

Actual record contains key

Clustering

- (clustered index: actual data is roughly sorted in order of search key
 - inner nodes correspond to leaf nodes on left and so on
 - 1 I/O per page of records
- Unclustered index: data not in order of search key
 - 1 I/O per record

Cost to full scan B+ tree = # data pages

Disk vs Memory

Disk - slow, lot of space

Memory - fast, limited space

Read/Writes - in memory

Use Disk for data pages not needed immediately

Buffer Manager

- manages which pages are loaded in memory
- controls when pages are read from and written to disk
 - decides pages to evict based on page replacement policy

Replacement Policies

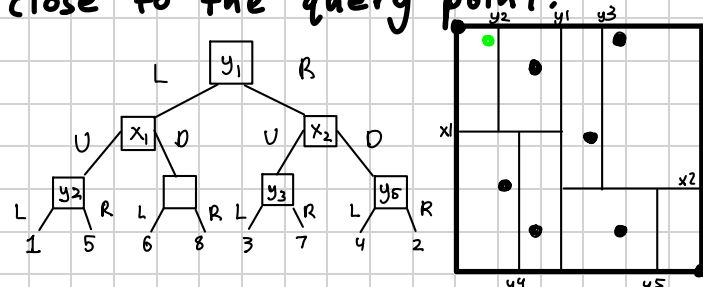
LRU - evict least recently used pages, good when certain pages are popular.

Clock - arrange frames in circle. Pass in a circle, if ref bit is 1 set to 0, if it's 0, you can evict. If it's pinned just skip it. Do not move clock hand if hit.

MRU - evict most recently used page, good for sequential scans

Spatial Indexes $O(\log N)$

K-d trees - used to perform nearest neighbor and range queries over high dimensional data. Partitions points into sep boxes. Nearest neighbor search involves searching boxes close to the query point.



External Algorithms

- designed for the case that there is more data than space in memory
- Can't access values whenever, disk accesses are expensive.
- Strategy: Divide and Conquer
 - Start w/ chunks that do fit in memory
 - Sort using External Merge Sort

General External Merge Sort

- Efficiently sort N pages with B buffer pages in memory
- Use a conquer and merge strategy w/ n passes: $\lceil \log_{B-1}(N/B) \rceil$ runs of B pages
 - Pass 1: sort more pages at once, so fewer runs to merge
 - can sort B pages at once
 - Pass 2- n : merge more runs at once, so finish faster
 - can merge $B-1$ runs at once $\rightarrow \lceil \log_{B-1}(B-1) \rceil$ runs from prev pass

I/O Cost = $2N * (1 + \lceil \log_{B-1}(N/B) \rceil)$ 2nd term total #passes

Hashing

- Want to be able to group together tuples with the same key value
- Partition the data with hash functions applied on the key
 - all tuples w/ a certain key will be in the same partition
- Useful for removing duplicates, grouping data

External Hashing

$3[R] + 2[S]$ for each extra pass

- Use a hash function h_p to partition the data.
- Write the partitions to disk
- (can split into $B-1$ partitions)
- If the partitions are small enough to fit in memory ($\leq B$ pages), load them in and make an in-memory hash table for each one, one at a time.
 - Use a different hash function, h_r , that is independent of h_p .
- Assume perfect hash functions (equal partitions)
- Recursive partitioning keeps applying hash functions until partitions fit in memory
- All hash functions must be indep.
- Terminate algorithm if all values in partition are the same.
- At each pass, you write all the orig pages to disk + $B-1$ pages per partition.
- Partition is done when $\leq B$ pages while being read. Write it out and don't read it next time.

Iterators

- No support for random accesses
- pulls records one at a time
- Stream: records read one by one (filter)
- Materialize: Operator must see all input before producing output. (sort, hash)

Joins

- Take one relation and matching each tuple with tuples from another relation
- The join condition determines what rows in the other relation match to a row in the first relation
- Exclude the final write's I/O cost, might not be materialized

Notation

$[R]$ = # of pages in R
 P_R = # of records per page in R
 $|R|$ = # of records in R (cardinality)
 $|R| = P_R * [R]$

Simple Nested Loop Join (SNLJ)

$R \bowtie S$:

```

for row r in R:
  for row s in S:
    if  $\theta(r,s)$ :
      output r joined w/ s

```

I/O Cost = $[R] + |R| * [S]$

- Flipping order can change I/O cost.

Page Nested Loop Join (PNLJ)

$R \bowtie S$:

```

for page  $P_R$  in R:
  for page  $P_S$  in S:
    for row r in  $P_R$ :
      for row s in  $P_S$ :
        if  $\theta(r,s)$ :
          output r join s

```

I/O Cost = $[R] + [R][S]$

Block Nested Loop Join (BNLJ)

$R \bowtie S$:

```

for block of  $B-2$  pages  $C_R$  in R:
  for page  $P_S$  in S:
    for row r in  $C_R$ :
      for row s in  $P_S$ :
        if  $\theta(r,s)$ :
          output r join s

```

before selectivity $\rightarrow$ after selectivity

I/O Cost = $[R] + \lceil [R]/(B-2) \rceil [S]$

Index Nested Loop Join (INLJ)

$R \bowtie S$: index on S

```

for row r in R:
  for row s in S that satisfies  $\theta(r,s)$  (found using the index):
    output r join s

```

I/O Cost = $[R] + |R| * \text{cost to find matching S tuples}$
 Cost to find matching S tuples:
 Alt 1: Cost to traverse root to leaf + read all leaves w/ matching tuples
 Alt 2/3: Same as Alt 1 + 1 I/O per page if clustered, else, 1 I/O per tuple

Sort Merge Join (SMJ)

1. External Sort R and S by the join key.
2. Iterator on R , Iterator on S .
 If R 's key $<$ S 's key, advance R .
 If S 's key $<$ R 's key, advance S .
 If keys are equal, output match
3. Mark first match s of any row r in R . Continue iterating as normal, when R advances, if the key of r is the same, reset S to the mark. Otherwise, release the mark.

I/O Cost = sort R + sort S + $[R] + [S]$
 Optimized: Stream sorted output into merge instead of materializing
 - Saves $2[R]$ or $2[S]$ or $2([R] + [S])$
 - Only works w/o duplicates
 I/O Cost = sort R + sort S - $[R] - [S]$

Buffer req to stream both:
 $\text{runs}(R) + \text{runs}(S) \leq B-1$

Buffer req to stream one:
 $\text{runs}(R) \leq B-2$

Grace Hash Join

1. Hash both R and S using hash function h_b into $B-1$ partitions. Write all partitions to disk.
2. For each partition pair (R_i, S_i) :
 Load R_i into memory and build an in memory hash table using h_r . Stream in tuples of S_i , probe the in memory hash table, output matching tuples.

Partitions of R must fit in $B-2$ pages. 1 to stream S partition 1 for output.
 If S is smaller do $S \bowtie R$. Also can recursively partition.

Relational Algebra

- always closed (always returns relation)

Unary Operators:

Projection (π): $\pi_{col1, col2}(R)$ returns R but with only $col1$ and $col2$ like a SELECT.
 - If you project away the columns that 2 tuples differ by, they collapse into one tuple.

Selection (σ): $\sigma_{pred}(R)$ keeps only tuples from R that satisfy pred. $\rightarrow$ commutative
 - same idea as WHERE

Renaming (ρ): $\rho_{id \rightarrow sid}(R)$ returns R but w/ the id attribute renamed to sid.
 - SELECT id AS sid

Binary Operators:

Union ($\cup$): $R \cup S$ contains all tuples in R and all tuples in S . Only defined when both relations have the same attributes.
 - if a tuple appears in both R and S , collapses into 1.
 - same as UNION

NLJs: null check if right has Next store right record, compare to left if 0, L.concat(r) else if left has more, advance left, reset right. else, return null

Parent	Child
NL	Nothing
S	Nothing
X	Nothing
IS	IS, S
IX	IX, X, IS, S, SIX
SIX	IX, X

Multi-granularity Locking

- allow for locking at diff levels (DB, table, page, tuple)

Intent Locks

- To S lock tuple, IS DB, table, page first. To X lock tuple, IX DB, table, page first

An IX lock doesn't prevent placing S lock on a diff tuple, but an IS lock prevents you from placing an X lock on tuple.

SIX lock combines S and IX to scan table, update page, etc. Remove all S and IS locks that are descendants of the SIX lock.

Set Difference ($\rightarrow$): $R \rightarrow S$ returns R but with any tuple that appears in S removed. Only defined when R and S have the same attributes

- EXCEPT

Cross Product ($\times$): $R \times S$ returns a relation with a tuple (r, s) for every tuple r in R and every tuple s in S .

- Defined even when R and S have diff attributes $R \times (S \cup T) = (R \times S) \cup (R \times T)$

Compound Operators:

Intersection ($\cap$): $R \cap S$ returns a relation that has only tuples appearing in both R and S .

- Only defined when R and S have the same attributes.
- Equal to $R - (R - S)$

Theta Join ($\bowtie_\theta$): $R \bowtie_\theta S$ returns the inner join of R and S on the join condition θ

- Equal to $\sigma_\theta(R \times S)$
- $R \text{ INNER JOIN } S \text{ ON } \theta$

Natural Join ($\bowtie$): $R \bowtie S$ returns the natural join of R and S , which is on every column with the same name.

- $\sigma_{R.col1=S.col1 \dots R.colN=S.colN}(R \times S)$

Group by / Aggregation (γ):

- $\gamma_{col, COUNT(*) > 2}(R)$ returns R grouped by col , and filters groups that don't have $COUNT(*) > 2$
- $\gamma_{MAX(col)}(R)$ returns max over R on col
- $\gamma_{col, AVG(col2), COUNT(*) > 2}(R)$ is the same as $SELECT col, AVG(col2) FROM R GROUP BY col HAVING COUNT(*) > 2$

Query Optimization

Naive plan: scan A , scan B , join, select

Alternate plans:

- ① Push selections/projections down the tree.
- ② Materialize intermediate relations (write to a temp file)
- ③ Use indices

BNLJ/PNLJ Only SNLJ, INLJ, and SMS preserve interesting orders.

If σ on left relation, pushing down helps, otherwise doesn't help.

If materialized, pay write cost once, but reduced size for future scans.

Total Cost = scan costs + materialize cost + join cost of pages that are fed.

Plan Space = Set of plans considered by optimizer

Cost estimation: IO cost per plan

Search Algorithm - how to efficiently find best plan

Selectivity Estimation

Selectivity - fraction of tuples satisfying a predicate (0 to 1)

- default is $\frac{1}{N}$
- Assume data is uniformly distributed across the range of values it takes on
- Assume predicates on different columns are independent of each other
- Predicates on the same column are often dependent like $x < 10$ and $x > 20$.

Let $|c|$ = # of distinct values in column c

If you have index on column, assume you know $|c|, \max(c), \min(c)$

output tuples/records = $|(\text{selectivity})(\text{total \# of tuples})|$

Pred	Selectivity
$C = V$	$\frac{1}{ c }$ if you know $ c $
$C1 = C2$	$\frac{1}{\max(c1 , c2)}$ if you know both
$C < V$ integer	$\frac{1}{ c }$ if you know one
$C > V$ integer	$\frac{V - \min(c)}{\max(c) - \min(c) + 1}$
$C \leq V$ integer	$\frac{\max(c) - V}{\max(c) - \min(c) + 1}$
$C <= V$ integer	$\frac{V - \min(c) + 1}{\max(c) - \min(c) + 1}$
$C >= V$ integer	$\frac{\max(c) - V + 1}{\max(c) - \min(c) + 1}$
$C >= V$ float	$\frac{\max(c) - V}{\max(c) - \min(c)}$
$C <= V$ float	$\frac{V - \min(c)}{\max(c) - \min(c)}$
$p1 \text{ AND } p2$ independent	$S(p1) \cdot S(p2)$
$p1 \text{ OR } p2$	$S(p1) + S(p2) - S(p1 \text{ AND } p2)$
NOT p	$1 - S(p)$

Build a graph with 1 node per transaction. If an operation in T_i conflicts w/ an operation in T_j , and the op in T_i comes first, add an edge from T_i to T_j . All conflict serializable schedules have an acyclic dependency graph. Just check for cycles.

Selinger Optimizer

Plan Space: Only left deep trees, avoid joins with no join cond

Search Algorithm: Dynamic Prog.

Left Deep: $((A \bowtie B) \bowtie C) \bowtie D$

Runtime: $O(n \cdot 2^n)$

Interesting Order: If query has ORDER BY, GROUP BY, or a JOIN later, might be worth to sort by that attribute now.

Drop plans if no join cond.

Pass 1: For each (relation, interesting order) pair, find min cost access method.

Drop plans if they don't have an interesting order AND it's not the cheapest of the relation

Pass i: For each (subset of size i of relations, interesting order), find min cost access method.

Drop plans if they don't have an interesting order AND it's not the cheapest method for the subset. Interesting orders expire, if interest comes from $S \cdot x \leq T \cdot x$ but you are joining S & T

$\{A, B\} = \{B, A\}$ but $A \text{ BNLJ } B$ and $B \text{ BNLJ } A$ might have diff costs.

Transactions

Collections of operations that are a single logical unit.

- We want all operations to happen at once.

Atomicity - All operations in a trans happen or none of them.

Consistency - Transaction must leave DB in a state that satisfies all rules of the DB

Isolation - If two transactions run concurrently, they shouldn't interfere w/ each other.

Durability - A committed transaction should persist.

Commit - successful transaction (save changes)

Abort - unsuccessful (revert changes)

Serializability

Schedule - order in which operations of a set of transactions are executed.

Serial Schedule - every transactions runs start to finish without interleaving.

Equivalent schedules involve same transactions, final state is the same, and each transaction has the same order of ops in both.

Isolation $\rightarrow$ schedule is equivalent to a serial schedule.

We check if schedules are conflict serializable.

Conflict if ① At least one operation is write

② diff transactions

③ work on same resource

Conflict Equivalent: Let S_1 have conflict where $T_1 R(B)$ comes before $T_2 W(B)$. Then S_2 will also have $T_1 R(B)$ before $T_2 W(B)$. This must apply for all conflicts.

If conflict equivalent to serial schedule $\rightarrow$ conflict serializable